

Fog of War without Zero-Knowledge Proofs: A Blinded Mutual-Sighting Test for Trustless Game Channels

Andrew Colosimo Roy Crombleholme Andrew Gore
Konstantin Gorskov Daniel Kraft

Xaya Developers

July 31, 2026

Abstract

Replicated deterministic state machines, blockchains and the state channels built on them, make hidden information awkward: every replica must be able to verify every transition, yet fog of war requires that positions stay secret. Commit and reveal hides positions but destroys *encounters*: two players walk past each other in the dark because neither may learn that the other is near. We present a *blinded mutual-sighting test*. After both players have hash-committed their moves for a round, and before either move is disclosed, the two clients run a Diffie–Hellman-blinded private membership test, the Meadows 1986 and Huberman–Franklin–Hogg 1999 lineage cut down from set intersection to a single membership bit evaluated in both directions. Both parties learn exactly one shared bit, “we are in sight”, and nothing else. Three design choices adapt the classical protocol to a consensus setting. First, every blinding scalar and pad element is derived deterministically from per-epoch secrets the player is already bound to reveal, which makes the honest transcript recomputable by an after-the-fact audit — and any reveal that publishes a secret closes its epoch on the spot: the referee restarts the player’s commitment chains and the client re-seeds behind them, so a disclosed secret never governs a live round. Second, no conviction ever trusts the exchange itself: cheating is adjudicated by recomputing the public sight predicate over positions fixed by commitments made before the round’s information existed. Third, the blinded query element each round publishes is bound, retroactively, to the position committed beside it — one sampled curve check at the audit, plus a pay-on-accusation dispute over sealed epochs — pricing, at audit strength, the chosen-element oracle that unbound private-membership exchanges leave open. Together these replace an in-round zero-knowledge proof, which does not fit the execution budget of our referee, with after-the-fact attribution. On the honest path the referee performs no curve arithmetic — its in-round work is refusing a published set that does not rebuild the committed root, which is hashing; audits and disputes each buy a fixed, measured handful of curve operations — and the chain is touched only to open, dispute and settle.

1 Introduction

Multiplayer games on public replicated state machines inherit a contradiction. The chain, or the channel, exists so that every participant can verify every transition; fog of war exists so that participants *cannot* see part of the state. The standard resolution, commit now and reveal later, solves only half of the problem. A hash commitment hides a player’s position perfectly, and perfect hiding is precisely what makes the game unplayable: if neither player may learn anything about the other’s position before the endgame, then two players a few tiles apart walk past each

other in the dark, every round, for the whole match. In our own testbed, removing all position disclosure produced long stretches of mutual line of sight in which neither player was ever told anything, which is an invisible non-game.

The requirement is therefore a conjunction:

1. **Privacy.** Nobody learns anything about your position unless they are close to you, or unless you spend it deliberately, for instance on a contested objective.
2. **Encounters.** Two players who *are* close must both learn so, in the same round, every round, with no exceptions.
3. **Adjudication.** Every rule above must remain enforceable by a referee that never sees secret state, on a metered execution budget.

The natural cryptographic answer to requirement 3 is a zero-knowledge proof attached to every move. That answer is unavailable to us in-round: our referee is a zero-import WebAssembly blob with a hard fuel budget per call, and a single Groth16 verification costs more than the whole budget. This paper describes what we built instead.

Contributions.

1. A *blinded mutual-sighting test* (§5): a two-party private membership test, run in both directions between move commitment and move disclosure, that yields the one shared bit $\text{sight}(a, b)$ and nothing else. The construction is the classical Diffie–Hellman private-matching protocol of Meadows and of Huberman, Franklin and Hogg [1, 2], specialised from $\text{set} \cap \text{set}$ to membership of a single element.
2. A *determinism-for-auditability* discipline (§7): every byte a party should send is a pure function of secrets it is already committed to reveal, scoped to an *epoch* that any reveal terminates (§4.2), so a judge can recompute any disputed transcript after the fact and no reveal ever un-blinds a live round. This converts “prove your input was honest”, which is a proof system’s job, into “lying is non-repudiable evidence”, at negligible consensus cost.
3. *Enforcement by geometry, not cryptography* (§7): no conviction ever depends on the exchange. Suppressing a sighting, and disputing a phantom one, are both adjudicated by recomputing the public sight predicate over positions bound by hash commitments made before the round’s information existed.
4. A *binding for the blinded query element* (§7.1): the one input the exchange cannot validate in-round — a responder able to tell what it is being asked would defeat the instrument — is made retroactively checkable against the committed position instead, by a sampled check at the audit and a dispute over sealed epochs in which every well-formed filing convicts one of its two parties (a malformed filing is refused without a verdict). We measured the oracle this prices and found it decisive rather than marginal (§6.1).

2 Setting and threat model

Game channels. The system runs on *game channels* [21] as deployed on the Xaya platform: two players play a turn-based game off-chain, exchanging signed moves, with the blockchain as the

fallback arbiter. A single host process referees many different games by loading each game’s rules as a hash-pinned, zero-import WebAssembly blob; the blob is fuel-metered per call and its state is size-capped. Both clients and every potential arbiter run the identical blob, so “the judge” below means *any* party replaying the rules; honesty of the judge is by replication, not by assumption.

Round structure. Play proceeds in rounds on a fixed public map, in our deployment a tile dungeon generated procedurally from a shared seed. Each round:

1. **Commit.** Both players publish hash commitments to their move, position and action, salted from a per-epoch path secret (§4.1). The same move carries a Merkle root over the flight the player is about to publish, with its own blinded element beside it in the clear — the ordering §7 depends on: every root is public before any flight exists on the wire. The referee also appends that blinded element to an append-only Merkle frontier carried in the player’s state, the retention behind §7.1.
2. **Sighting test.** The exchange of §5, run as a pass of its own: each player publishes its blinded set and its responses as signed channel moves, and the referee refuses a published set that does not rebuild the root committed in step 1. Responses are computed from the peer’s committed element read out of the published state, so no message is delivered outside the channel.
3. **Disclose.** If the test said *no*, a player publishes nothing about its position. If it said *yes*, both players are obliged to publish their exact positions for the round, with an arrival proof, and on entering contact the path connecting the position to the player’s last judged one.

Not every ending runs an audit. A match decided during play finishes on the spot: by a kill (combat is legal only in contact, entering which requires a verified full-path proof, so the fight’s participants already stand audited), by a forfeit, or by a conviction that leaves one player standing. The round cap, and any player demanding it, close instead with an **endgame audit**: each player reveals its current secrets and its path back to its last *epoch boundary* (§4.2), which the blob verifies against those rounds’ commitments and replays against the map. Rounds before a boundary were verified identically at the moment their epoch closed, and the closure is pinned into the state by preimage: no round of movement is verified twice, and none that a verdict leans on escapes verification. Mid-match reveals — first contact, dispute evidence — are epoch boundaries rather than exceptional events. The audit predates the sighting test; §7 extends it.

Adversary. A malicious client controls all of its own messages: it may send arbitrary bytes in the sighting exchange, stall, or disclose falsely. It cannot break hash commitments or signatures, and it cannot dodge an audit once one is owed except by forfeiting, since timeouts convert stalling into forfeit. Both players’ clients may be malicious simultaneously; the game between two colluding cheaters is not our concern, but neither cheater may gain against an honest opponent. The design rule throughout: *consensus never trusts a claim*, and everything that convicts is recomputed by the blob from committed data.

3 The sight predicate

Positions are walkable tiles of the public map. For tiles a, b let $d(a, b)$ be the shortest walkable path distance and let $\text{los}(a, b)$ be the game’s line-of-sight predicate, a symmetric visibility test on

the tile grid. For a fixed sight range ρ the predicate jointly computed each round is

$$\text{sight}(a, b) := [d(a, b) \leq \rho] \wedge \text{los}(a, b). \quad (1)$$

Here d is symmetric by construction, and los was verified symmetric exhaustively over all in-range walkable pairs of the test maps. Symmetry makes “the” shared bit well defined: writing

$$V(x) := \{t \text{ walkable} : \text{sight}(x, t)\}, \quad a \in V(b) \iff b \in V(a), \quad (2)$$

with $x \in V(x)$ always, since distance 0 satisfies (1). The protocol pads every set to a fixed size N chosen above the largest $|V|$ the map admits (§4.1), so message size and work are constants independent of the sender’s surroundings.

The predicate must be symmetric for the construction to apply at all: asymmetric sight ranges would make the two parties’ bits legitimately disagree and the shared-bit framing collapse. Consequently the game defines exactly one sight range in consensus, and light sources are cosmetic.

4 Cryptographic requirements

The protocol is not tied to a particular group. It needs three properties, and only the third is easy to get wrong.

1. **Commuting blinding.** A group operation and scalars for which $\alpha(\beta X) = \beta(\alpha X)$. Any prime-order group serves, and so does RSA-style multiplicative blinding; nothing below depends on the choice.
2. **Hardness.** Distinguishing $\{\alpha P_t : t \in T_0\}$ from $\{\alpha P_t : t \in T_1\}$ for candidate sets T_0, T_1 , given the public P_t , must be hard. In a prime-order elliptic-curve group this is a standard decisional Diffie–Hellman problem — for a passive observer of the transcript. A *responding* participant additionally obtains one blinded evaluation αQ per round for a Q it supplies, a one-query oracle for $x \mapsto \alpha \text{H2C}(x)$, so claims against a receiver rest on assumptions of the one-more or gap Diffie–Hellman kind [7, 8]; §6 scopes which claim rests on which.
3. **Elements with unknown discrete logarithms.** The map $t \mapsto P_t$ from a game position to a group element must be a hash-to-group map whose outputs have discrete logarithms unknown to everyone, including the party doing the hashing. This is load-bearing rather than stylistic, for the reason given in §6.

Our instantiation. We use Curve25519 [17] with x-only Montgomery arithmetic, the RFC 7748 [19] ladder, and an x-only Elligator 2 [18] map with the cofactor cleared at construction so that every point lies in the prime-order subgroup. The map is a single-map encoding rather than the RFC 9380 [20] suite: the protocol needs determinism, unknown discrete logarithms and prime-order outputs; it does not need the output to be indistinguishable from uniform, because every transmitted element is additionally randomised by a per-round blinding scalar. Interoperability is a non-goal, since every party runs the same pinned blob. The choice matters only for cost, and the reference implementation [16] carries the details, the known-answer vectors, and the argument for each deviation.

4.1 Derivations: nothing is random

Every quantity in the exchange is derived rather than sampled. This is the part that makes the audit of §7 possible, so it is a protocol requirement and not an implementation detail:

$$\text{position elements (public): } P_t = \text{H2C}(\text{"T"} \parallel \text{enc}(t)), \quad (3)$$

$$\text{blinding scalar of player } i, \text{ round } r: \alpha_r^{(i)} = \text{SHA-256}(\text{"P"} \parallel \text{seed}_i^{\text{pet}} \parallel \text{enc}(r)), \quad (4)$$

$$\text{pad dummies of player } i, \text{ round } r: D_{r,j}^{(i)} = \text{H2C}(\text{"D"} \parallel \text{seed}_i^{\text{pet}} \parallel \text{enc}(r) \parallel \text{enc}(j)), \quad (5)$$

with distinct domain-separation tags, the pad index running $j = |V|, \dots, N - 1$, and with α reduced into the scalar range and the measure-zero zero case patched deterministically. Here $\text{seed}_i^{\text{pet}}$ is one half of a split pair of secrets, defined next.

The dummies of (5) are deliberately *round-dependent*, and we know the property is load-bearing because an earlier revision lacked it. A dispute filing publishes that round's α_r by design (§7), and α_r is invertible modulo the group order, so an observer can divide the filing round's published set by it and recover the raw pad elements. For the filing round itself this is harmless — a filing publishes the filer's exact position anyway, which dominates $|V|$ — but pad elements fixed per seed would remain in service in every later round, recognisable there through the chosen-element channel of §6.1: each recovered element, replayed as a query, yields a threshold bit about the current $|V|$ (the pad fills the slots above $|V|$, so “still in use” bounds it by that element's index) — and with a fixed pad it would keep yielding that bit in every later round, a standing oracle handle — $|V|$ being the one quantity the fixed-size pad exists to hide. Folding the round into the derivation confines any recovery to the round already disclosed. The cost is that only the position elements of (3) can be precomputed; the pad is re-derived each round, the same N hash-to-group maps the budget already prices.

The two-secret split. The secrets are two, and the distinction is load-bearing. Player i holds one master secret and derives

$$\text{seed}_i^{\text{path}} = \text{SHA-256}(\text{"A"} \parallel \text{master}_i), \quad \text{seed}_i^{\text{pet}} = \text{SHA-256}(\text{"B"} \parallel \text{master}_i), \quad (6)$$

the first salting every move commitment, the second — the *PET seed*, after the private equality test at the exchange's core — feeding (4) and (5) and nothing else. A full-path reveal publishes both: the path seed, which is what the movement audit needs, and the PET seed of the epoch that same transition closes — the mint of §4.2 is the transition's final step, so a disclosed PET seed never governs a round still to be played. (The only reveal that carries no PET seed is the empty one from a player whose restart was minted that very round.) The one scalar a dispute's adjudication needs rides the filing itself, unproven, and is pinned there by the adjudication's own equalities (§7). Both sub-seeds are hash images, so publishing them reveals nothing about the master and hence nothing about any later epoch. The reuse claim above survives in scoped form: the honest transcript of an epoch is a pure function of that epoch's two seeds, the committed positions and the public map, and the player is bound to expose exactly those, in the reveal that ends the epoch.

An earlier revision derived salts and blinding scalars from one seed, and the coupling was a live defect rather than a stylistic blemish: the protocol's own disclosure rule forces a full-path reveal at first contact, mid-match, after which the one published seed made every past and future α_r computable — the player's published Q_r then decodes to its exact tile by testing the few thousand candidates, every remaining round. The split, with the epoch machinery below, is what makes the privacy claims of §6 true as stated rather than true only until first contact.

4.2 Epochs: a published secret is a spent secret

Full-path reveals are not confined to the endgame. The disclosure rules force one at first contact — a player entering contact without a standing proof must ship the connecting path, verified in that same call — and a sight dispute’s filing always carries one as its evidence, even from a filer already proven, because the adjudication anchors on the filer’s judge-verified tile for the disputed round. (The sealed-epoch dispute of §7.1 carries none — its evidence is entirely public — and is not an epoch boundary.) Each such reveal publishes the revealer’s path seed, and a published path seed opens every commitment it will ever salt, since a round’s committed tile and action span under 2^{19} candidates. A protocol that continued play under a published seed would play the rest of the match face-up.

The referee therefore *mints a restart* as the final step of any transition that accepts a full-path reveal from a living player in an unfinished match. The mint folds each of the player’s commitment-chain heads into a fresh one with the old head inside the preimage, so the verified prefix stays pinned and a restart can never shed history; it records the just-verified position as the new epoch’s anchor, so a restart is not a licensed teleport; and the client re-seeds behind it,

$$\text{master}'_i = \text{SHA-256}(\text{"E"} \parallel \text{master}_i \parallel \text{enc}(R)), \quad (7)$$

re-deriving both sub-seeds from the new master by (6). The master never appears on any wire, so the ratchet moves only forward: nothing published in one epoch says anything about the secrets of the next.

The stretch between boundaries is an *epoch*, and every audit is epoch-relative: a reveal covers exactly the rounds since the revealer’s boundary and anchors to the boundary’s judged position. Three consequences matter below. A mid-match reveal costs exactly the epoch it closes — which that reveal published anyway — and nothing after it. No round is ever verified twice, so the total audit work over a match is linear in its length rather than quadratic; a single audit call still pays for the epoch it opens, which in the worst case is the whole match. And a dispute among more than two players is no longer terminal for the table: the audit pass it forces takes reveals only from the accused — the filer’s proof was parked at its filing — convicts on the evidence, and returns the survivors to play with the spent epochs re-seeded. (The demand-an-immediate-end challenge move remains terminal by design; a challenge the match could continue past would be a purchasable snapshot of every player’s position.)

5 The protocol

Fix round r ; both players have committed position and action for r . Write $\alpha_A = \alpha_r^{(A)}$, and let a, b be the committed positions. Before any disclosure:

Flight 1 (both directions).

$$A \rightarrow B : \quad S_A = \text{sort}\left(\left\{ \alpha_A P_t : t \in V(a) \right\} \cup \left\{ \alpha_A D_{r,j}^{(A)} : j = |V(a)|, \dots, N-1 \right\}\right), \quad Q_A = \alpha_A P_a, \quad (8)$$

and symmetrically $B \rightarrow A$ sends (S_B, Q_B) . Here S is N elements plus the single element Q , and sort orders by encoding, a canonical order independent of enumeration that reveals nothing about which entries are dummies. Exactly $N+1$ hash-to-group maps and $N+1$ scalar multiplications are performed whatever $|V|$ is, so neither length nor timing leaks the sender’s surroundings.

Flight 2 (both directions).

$$A \rightarrow B: \quad R_A = \alpha_A Q_B (= \alpha_A \alpha_B P_b), \quad B \rightarrow A: \quad R_B = \alpha_B Q_A (= \alpha_B \alpha_A P_a). \quad (9)$$

Local finish.

$$\text{bit}_A = [R_B \in \{\alpha_A s : s \in S_B\}], \quad \text{bit}_B = [R_A \in \{\alpha_B s : s \in S_A\}], \quad (10)$$

each costing N scalar multiplications and N comparisons.

Theorem 1 (Correctness). *Except with probability negligible in the hash length, $\text{bit}_A = \text{bit}_B = \text{sight}(a, b)$.*

Proof sketch. On the prime-order group, $\alpha_A(\alpha_B P_t) = \alpha_B(\alpha_A P_t)$, so $R_B = \alpha_A \alpha_B P_a$ equals $\alpha_A(\alpha_B P_t)$ if and only if $P_a = P_t$, that is if and only if $a \in V(b)$, unless two distinct hash-to-group inputs collide, which is bounded by the birthday term over the domain-separated inputs. Thus $\text{bit}_A = [a \in V(b)]$ and symmetrically $\text{bit}_B = [b \in V(a)]$, and these coincide with $\text{sight}(a, b)$ by (2). With x-only encodings P and $-P$ are identified, but hash-to-group outputs are themselves x-coordinates, so that coincidence is the same collision event and is already counted. $\square$

The deployed client asserts that the two bits agree on every round for which it can observe both, and treats disagreement as a defect rather than as an input.

Reference implementation. An open-source implementation of the protocol, together with an independent reimplementaion written against the protocol’s specification in a separate codebase with no shared code, is available [16]. The two reproduce each other’s flights, responses and finishing bits byte for byte, so the specification and the deployed system cannot silently diverge.

6 Security analysis

The designed leakage is the bit, and, by padding, nothing about $|V|$.

The blinded set hides its contents — stated with its hypothesis. Recovering anything about $V(a)$ from S_A alone is the distinguishing problem of §4, requirement 2. The receiver, however, holds more than S_A : it holds $Q_A = \alpha_A P_a$, and the response $\alpha_A Q_B$ hands it one evaluation of $x \mapsto \alpha_A \text{H2C}(x)$ per round at a point of its choice — for a protocol-following receiver that point is its own committed tile, which is exactly what (10) consumes. Set privacy against a receiver is therefore a one-more/gap Diffie–Hellman-style claim (§4): the one evaluation tells the receiver whether its queried point lies in the set, by design, and must help with nothing else. A receiver that *deviates* spends the same one query on an arbitrary point instead; §6.1 measures what that is worth, and §7.1 is what the deployed system does about it.

Unknown discrete logarithms are load-bearing. The blinding must be by a secret factor *and* the elements being blinded must have unknown discrete logarithms. If positions were instead encoded as $P_t = h_t G$ with h_t a public hash of t , then every element’s discrete logarithm would be public, and a receiver of one flight could strip the blinding: each h_v is public and invertible, so multiplying a received element by h_v^{-1} for candidate positions v yields αG at the correct guess, after which one further pass labels every element of the set. Over a game’s small universe of

positions this is a few thousand scalar multiplications, so the sender’s set, and hence its position, is recovered from a single honest protocol run. With a hash-to-group map the corresponding first step, obtaining αG from αP_t without knowing $\log_G P_t$, is the computational Diffie–Hellman problem. This is also why brute force over a small universe buys nothing: “is this element position t ?” has no checkable relation without breaking Diffie–Hellman.

Remark 1. This is not a hypothetical. A published reference implementation of ECDH-PSI constructs its elements exactly this way [13]; we reported it upstream, and it was fixed. It is the sharpest illustration we know of why the hash-to-group map is not a stylistic detail.

Cross-round unlinkability. α_r is fresh per round and derived from a secret that leaves the client only after the epoch it governs is closed (§4.2), so linking $\alpha_r P_t$ to $\alpha_{r+1} P_t$, a stationary player, is again a Diffie–Hellman distinguishing problem for an observer of the honest transcript. The pad dummies are re-derived each round for exactly this reason: elements recurring across rounds would become a linkable tag the moment any one round’s scalar went public, and a dispute filing publishes one scalar by design (§4.1).

Reflection and replay. B may echo one of A ’s own set elements back as its Q_B , giving $R_A = \alpha_A^2 P_t$. Exploiting the pair $(\alpha_A P_t, \alpha_A^2 P_t)$ is the square Diffie–Hellman problem, for which no attack is known in the groups we use. The bit that B computes from a garbage Q is garbage, which is self-sabotage, and it becomes attributable evidence under §7.

Malformed elements. A malicious peer can substitute an arbitrary string for a group element. In our x-only instantiation the ladder processes off-curve and low-order inputs without special-casing; the worst case is a small-order element sent as Q , whose reply αQ reveals α modulo the cofactor, a handful of bits out of the scalar’s full width, and useless against the distinguishing problem. The reply never matches a cofactor-cleared set element, so the resulting bit is 0. Like every malformed flight this is self-sabotage now and evidence later.

6.1 The chosen-element oracle, and what it is worth

The deviation above deserves stating as an attack, because we measured it and the measurement moved us. A responder cannot validate the query element — a responder able to tell what it is being asked would defeat the instrument — so a malicious player may aim its round’s Q at *any* tile t : submit $Q = P_t$, read the peer’s response $\alpha_B P_t$, and test it for membership of the peer’s published set. Sight is symmetric, so the answer is “are you standing within sight of t ” — a locator query about a place the asker is nowhere near, answered truthfully by every peer, claiming nothing and filing nothing.

Two prices attach to it even without the binding of §7.1. The asker forfeits its own warning for the round — its instrument answers about t , not about itself — and none of the disclosure obligations move with the question, so an asker genuinely in sight of its victim is convicted for staying dark exactly like a deliberate suppressor. It is tempting to scope the leak by those prices and by its rate limit of one bit per peer per round, and an earlier internal review of this paper did. Simulation says otherwise: an attacker that probes only when its belief set *proves* no opponent can see it takes zero conviction risk and still cuts a walking hider’s median candidate set from several hundred tiles (seventy-five for an objective-running policy) to roughly ten, pinning it inside ten tiles on 49–71% of rounds where an honest hunter manages 16–26%; only a sprinting policy, which also probes least (a third of its rounds, against 39–78% for the walking policies), keeps its cover

in the hundreds. One *optimally chosen* bit per round keeps pace with a one-step-per-round target indefinitely — the fixed point of dilate-then-halve at one step a round is a sixteen-tile candidate set, and the simulated medians (8–11) sit at that order of magnitude; the same arithmetic gives 64 tiles at two steps a round, which is the regime the mode-hiding below forces. The lesson we record: counting a leak’s bits per round is not the same as simulating an adversary that spends them well.

The deployed response is two-fold. First, the wire no longer publishes the declared movement mode below contact, so the attacker must dilate its belief set by the maximum step count every round; the measured pinning rate collapses from the 49–71% above to under 7% — still a leak, no longer an operationally decisive one. (The same zero-risk attacker, retooled for the mode-hidden wire, probes on 28–87% of its rounds — the fraction the sampled check of §7.1 prices.) Second, the query element is bound, retroactively and at audit strength, to the committed position (§7.1), which turns the oracle from an unattributable capability into a self-incriminating one. Neither is an in-round proof, and §9 states the residue.

The accumulated-negative leak. Every “no” round tells a player that the opponent is outside its visible set, and these constraints accumulate. This is the designed privacy and liveness trade of the whole scheme rather than an oversight: any protocol that guarantees requirement 2 of §1 must leak at least the negatives, because a guaranteed encounter is exactly the absence of a false negative. We measured the residual anonymity set on our own maps against an exact Bayesian attacker granted every advantage, and it remains large enough to play with, but the figure is a property of a particular map, sight range and movement speed rather than of the protocol, and it must be re-derived for any other setting.

What the exchange deliberately does not provide. First, no in-round proof that V or Q was computed from the committed position: a malicious party can feed any set or Q it likes, and the surrounding system makes this pointless or attributable rather than impossible (§7, §7.1) — with two bounded residues, an over-reported set and the window before a chosen element’s epoch is audited, recorded honestly in §9. Second, no fairness or liveness: a party can stall mid-exchange, and the channel’s existing timeout forfeits cover it. Third, it requires the predicate to be symmetric (§3).

7 Attribution instead of zero-knowledge

In-round, the referee performs no curve arithmetic over the exchange: its one check is that a published set rebuilds the committed root, which is hashing, and the correctness of the exchange’s *content* is never judged before an audit or a dispute — where curve work is bought deliberately and by measure, one sampled check per audited epoch (§7.1) and a fixed handful of multiplications per dispute. The honest path never touches the chain — a channel consults it only to open, dispute and settle. Three facts combine into the enforcement story.

1. **The honest transcript is a pure function.** By §4.1, every byte party i should send in round r is $T(\text{seed}_i^{\text{path}}, \text{seed}_i^{\text{pet}}, r, \text{position}_i(r), \text{map})$, the seeds those of the epoch containing r (§4.2): the scalar and the pad from the PET seed, V from the committed position and the public map, the commitment salts from the path seed, and the order fixed by the sort.
2. **The audit reveals the preimages.** The pre-existing endgame audit forces every player to reveal its epoch secrets and its path, verified against those rounds’ hash commitments,

with closed epochs pinned by the mints that ended them (§4.2). Any judge can therefore recompute, after the fact, the exact bytes an honest party would have sent in any round, and compare them with what was sent. Each party publishes a commitment to the flight it sent inside its own signed move, which makes a wrong byte non-repudiable. What the judge may never do is *regenerate* an honest flight from the seed — a hundred-odd curve derivations, some thirty times its per-call budget — so each check works from supplied or published bytes instead. The one secret-derived quantity the adjudication needs, the disputer’s round scalar, rides the filing itself rather than being derived from any revealed seed; it arrives unproven, and safely so, because every equality it enters compares it against bytes its filer committed or published before the dispute existed, so a forged scalar convicts the filer. On the accused’s side the evidence is one Merkle-authenticated element of the committed set, and the recomputation a verdict costs the judge is one hash-to-group map and two scalar multiplications per arm — an amount chosen against the fuel budget, since a first draft that checked twice as much did not fit beside the audit in the same call. On the disputer’s side a single named element is not even possible: a dark opponent’s position is revealed only by the audit itself, after the evidence is frozen, and the test’s own privacy hides *which* element matched — so the disputer files its whole set, the referee refuses the filing unless it rebuilds the committed root, and the judge then locates the element itself by scanning bytes that are already bound. Using the commitment the sender already publishes, rather than a signature scheme inside the blob, is what keeps all of this affordable.

3. **Convictions never trust the bit.** “In sight and stayed dark” and “disputed a sighting that was not there” are both adjudicated by recomputing **sight**, Eq. (1), over positions bound by commitments made before the round’s information existed. The bit only tells honest clients when they are obliged to act; the map convicts.

So a proof system’s job, *prove your input was honest*, is replaced by *your input is a deterministic function of things you are already bound to reveal, and lying about it is signed evidence*. The guarantee is delivered after the fact instead of in-round, which a turn-based channel game can afford when any player can force the audit, and the consensus-side cost is essentially zero.

The trust boundary, stated plainly. Attribution closes a gap that is open in both directions. A malicious opponent who fabricates its flight can make an honest client’s instrument read a phantom “yes”, causing it to over-disclose or to file a dispute; and it can equally make the instrument read a false “no”, blinding the honest client so that it fails an obligation it was never told it had. The second is the more dangerous, because it is a winning strategy rather than a nuisance: blind an opponent every round, watch it approach unaware, then escalate and dispute it, and a referee that judged the contradiction on geometry alone would convict the blinded victim.

Both close only if the commitment to the flight PRECEDES the exchange, which is why it rides the commitment move rather than the disclosure: the commit pass closes before any set is published, so every participant’s root is public before any flight exists on the wire. The response needs no commitment of its own, since the audit recomputes it from the revealed secrets and the peer’s committed element; publishing it is what makes a doctored response attributable. What the adjudication then establishes, on the two positions the audit has verified, is which side’s committed bytes contradict the geometry: whether the disputer’s own set contained the element that would have informed its victim, and whether the accused’s set produced the hit the disputer reports. Convictions therefore follow from committed data and never from either party’s claimed

bit, which is precisely the quantity an attacker forges. A false accusation is not free, and neither is a blinding carried out through the committed set.

One further property is required, and it is worth stating precisely because it is a property of the transport rather than of the protocol: the bytes a party DELIVERS must be the bytes it is later judged against. An earlier version of our implementation exchanged the flights off-chain and bound the response only by publication in a disclosure — a disclosure ordered, by the dispute’s own filing rules, after the victim had already had to choose whether to reveal. A party could therefore deliver a doctored response, or a doctored set, off-chain while publishing the honest values, and the recipient could not distinguish doctored from honest without the sender’s secret scalar. What the audit can recompute is what SHOULD have been sent, never what WAS; delivery is unprovable without attribution on the delivered bytes, so no adjudication rule over off-chain messages closes it. The deployed protocol therefore delivers nothing off-chain at all. Both legs of the exchange are PUBLISHED as signed channel messages in a pass of their own, after every commitment is fixed and before any disclosure: the referee refuses a published set that does not rebuild the committed root, each response is computed from the peer’s committed element read out of the published state — so the exchange needs no delivered message anywhere — and delivery and publication coincide by construction, at which point the recomputation convicts a doctored response exactly as it convicts a doctored set.

7.1 Binding the blinded element

One input to the exchange is not covered by the story so far. The published set is bound to a pre-committed root; positions are bound by hash commitments; but the query element Q was, in the terms above, merely *published* — nothing tied its bytes to the tile committed beside it, and §6.1 measures what an attacker does with that freedom. The binding cannot be checked where Q is used: the responder must not learn what it is being asked, and the referee, which could know, cannot afford curve arithmetic in every round’s calls. So the binding is retroactive, in three parts.

Retention. Each accepted commitment appends its Q to an append-only Merkle frontier in the player’s state — nine levels at our round cap, 288 bytes — maintained beside the commitment chain and reset at each epoch boundary.

A sampled check at the audit. An audit reveal now also carries the epoch’s PET seed and one Merkle opening. The judge draws one round of the audited epoch, from a hash over the running heads of every player’s commitment and disclosure chains, salted by an accumulator over the seeds already revealed in the pass; for any given prober those inputs are pinned by moves other players made after its queries were already chosen, so it can neither steer nor predict the draw. The judge then recomputes α_r from the revealed PET seed by (4), recomputes $\alpha_r P_{t_r}$ from the revealed tile, and verifies the opening against the frontier. A player that probed on k of the epoch’s n rounds is caught with probability k/n — 28–87% of rounds for the zero-risk strategies of §6.1 on the deployed wire, lowered only by asking less. A mismatch *rejects* the reveal rather than convicting: nothing has pre-committed the PET seed, so a mismatch cannot be attributed to a dishonest Q rather than to a mis-supplied seed, and a conviction may not hang on that ambiguity. Rejection forfeits the prober all the same: its committed Q opens under no seed it could supply, since the probe point and the true tile point have no known scalar relation, so no spelling of its reveal passes and the timeout ejects it.

Sealing, so closed epochs stay disputable. The check above covers the epoch the audit opens; without more, an epoch closed mid-match would launder its probes. The canonical attack probes from the dark, walks into contact and kills — and the contact proof’s own mint closes the probing

epoch, after which no later call holds its tiles. The mint therefore folds what it is about to discard — the frontier root, a hash of the epoch’s verified tiles, the spent PET seed, the boundary round and the round count — into a per-player accumulator that is never reset; the ratchet (7) has already re-seeded behind it, so the disclosure compromises nothing after the boundary. From the moment the epoch-closing reveal publishes the spent PET seed, every opponent can verify every round of that epoch off the wire for free; detection is certain, and only the conviction needs a referee. A dedicated dispute move names one round of one closed epoch and ships the accused’s whole sealed history. The referee re-folds the accumulator from zero — a rebuild that misses the accused’s standing value convicts the *filer*, since every input is public and the rebuild deterministic — then performs the one curve check at the named round, convicting the accused on a mismatch and the filer on a match. Every well-formed filing convicts somebody; a dispute that could cost nothing would itself be a free probe. And conviction is sound here where the audit could only reject, because every PET seed folded into the accumulator was sealed there by the accused’s own judge-accepted reveal: the accused authenticated the very bytes that convict it.

Measured, on the deployed referee. The audit reveal with its sampled check is the system’s worst call at 84.8% of the per-call fuel budget; the mint’s fold adds about 1.4% of the budget to the call that pays it; the closed-epoch dispute, rebuilding a worst-case 383-round epoch and paying its curve check in a move of its own, runs at 40.0%; and the retained state fits a four-player match in 7,865 of the platform’s 8,192-byte state cap. The residues — one draw per audited epoch rather than a check per round, and the kill-decided match that no audit pass ever reaches — are stated in §9.

8 Related work

The Diffie–Hellman private-matching lineage. The double-blinding trick at the core of §5 is old and good. Meadows [1] used commutative blinding for cryptographic matchmaking in 1986, and Huberman, Franklin and Hogg [2] used the same structure for private community matching in 1999. The modern literature knows it as ECDH-PSI: Agrawal, Evfimievski and Srikant [6] formalised the commutative-encryption approach for set intersection across private databases, Jarecki and Liu [7, 8] gave it security against malicious parties under one-more-Diffie–Hellman-type assumptions, and De Cristofaro and Tsudik [9] linear-complexity instantiations. (The other main PSI family, oblivious polynomial evaluation over homomorphic encryption, begins with Freedman, Nissim and Pinkas [5], who position it explicitly against this lineage.) It runs at planetary scale today, with Google’s Password Checkup [11] and Apple’s PSI system [12] both descendants. Our protocol is this classical construction *reduced*: from $\text{set} \cap \text{set}$ to membership of a single element, evaluated in both directions, because the game needs one shared bit and not the overlap.

Private set intersection for fog of war. The closest prior art to this work is OpenConflict [14]. Motivated by a practical memory-snooping map hack against real-time strategy games, it distributes game state so that each client holds only what it is entitled to see, using a private set intersection over grid cells, and pads the sets so that unit counts do not leak. It reports 22 ms per synchronisation at the peak of a professional StarCraft II game, so the approach is fast enough for real time and not merely for turn-based play. The padding idea in §4.1 is theirs. Two things differ here. OpenConflict assumes that players cannot lie about their inputs, having neither a referee nor a commitment layer, whereas our setting is a consensus engine in which a client may submit anything at all and the entire question is what happens afterwards; and our predicate is a

joint sighting test adjudicated by a replicated referee, rather than a state-distribution mechanism among clients who are trusted to be passive.

Applying that line of work to a blockchain game, with positions hash-committed on chain and revealed at the end of the match for deterministic replay, was sketched by Thomson [15], who also stated plainly that the case of actively dishonest players remained unsolved. That is the gap §7 addresses: replaying positions and actions does not detect a fabricated *protocol* input, because nothing in such a scheme binds the exchanged messages to the positions later revealed. Deriving every blinding scalar from a committed seed is what closes it.

The implementation we started from. Our engineering entry point into this literature was Thomson’s open-source ECDH-PSI implementation [13], a clear reference implementation of the classical protocol with an encrypted-payload layer for revealing which elements matched. Studying and prototyping against it shaped what we kept and what we changed: we adopted an explicit hash-to-group map, for the reason in §6; dropped the payload-encryption layer entirely, since we must never learn *which* element matched and so have nothing to decrypt; fixed the set cardinality by padding, since otherwise $|V|$ leaks for free; and, most importantly, bound the inputs. Like most PSI reference implementations that one leaves inputs unbound, so a malicious party can use the protocol as a membership oracle over fabricated sets and query points, and our committed-seed determinism, audit and query binding (§7, §7.1) exist precisely to close that gap.

Private equality and proximity testing. Comparing secrets without revealing them goes back at least to Fagin, Naor and Winkler [3]. The closest prior art to our setting is Narayanan et al.’s private proximity testing [10], which reduces location proximity to private equality of grid cells for social location-sharing. Our test differs in the predicate, walkable-path distance plus line of sight rather than Euclidean grid overlap; in being mandatory and bidirectional every round, since their users choose when to query whereas our players are *obliged* to act on the bit and the obligation is enforced by consensus; and in the adversary handled, since their protocols assume honest-but-curious phones whereas our clients are fully malicious and handled by attribution.

Hidden information in adversarial games. Mental Poker [4] founded the field of playing hidden-information games without a trusted dealer. The modern blockchain line is exemplified by Dark Forest [22], which achieves fog of war on a public chain by attaching a zk-SNARK to every move, proving it valid relative to hidden coordinates. That is the “prove your input was honest” path we could not take, since our referee’s per-call budget is below the cost of a single Groth16 verification, and channel games settle on chains where verifier costs bind harder still. The determinism-plus-audit construction of §7 is our substitute, and it is available to any turn-based game that already ends in a full reveal. Note also the difference in what is proved: Dark Forest proves unilateral facts about one player’s hidden move, whereas a sighting is an irreducibly *joint* predicate over two players’ secrets, which is what forces a two-party protocol rather than a proof.

Infrastructure. Game channels are due to Kraft [21], deployed on Xaya, the successor of Huntercoin, to our knowledge the first blockchain to run a multiplayer game world fully on-chain, and whose scaling and privacy limits motivated this line of work. Curve25519 is Bernstein’s [17], the Elligator 2 map is from Bernstein, Hamburg, Krasnova and Lange [18], the ladder and its test vectors follow RFC 7748 [19], and RFC 9380 [20] is the general hash-to-curve framework.

9 Limitations and future work

1. **Attribution is deterrence, not prevention.** A participant who fabricates a flight learns the answer it wanted before anything convicts it, so the stake must exceed the value of the information. The window is bounded differently per input: a fabricated *set* is judged only inside a dispute’s adjudication, so its bound applies to fabrications that must surface in one, while a fabricated *query element* meets the sampled check with the per-epoch probability of §7.1 and the closed-epoch dispute with certain detection — but each of those lands after the epoch, never in the round the answer was read. Disputes are no longer terminal for the table at more than two participants (§4.2); at two they still are, since the adjudication always convicts one of the pair. The demand-an-immediate-end challenge and the round-cap audit remain terminal by design.
2. **An over-reported set is a bounded, conditionally priced probe.** A participant may pad its published set with positions it cannot see, up to the fixed pad size per round, forcing an exposure from whoever it guesses right about. Padding is provable after the fact from the revealed seed, and a phantom “yes” hands its victim a *winning* dispute — the adjudication’s poisoning arm convicts the padder from its own committed bytes — but the victim must choose to spend that dispute, and no deployed rule prices the probe when it does not. Pricing it unconditionally is future work.
3. **The query-element binding is retroactive and sampled.** Between commission and audit a chosen element reads its answers freely (§6.1); the live check draws one round per audited epoch, so a sparse prober trades catch probability against questions asked; and the dispute needs a living filer with a disclosure pass to file in. At three or four players a bystander files. At two players the canonical probe-then-hunt-then-kill run ends the match at the kill, before any audit pass exists and with nobody left to file, so it convicts nobody on-chain: the killer’s own final reveal publishes the epoch’s PET seed and path, and its committed queries are already on the wire, so the probing is publicly verifiable by anyone — evidence without a court. This dead-victim residual is the sharpest gap the binding leaves. An in-round well-formedness proof for Q — a zero-knowledge statement of $Q = \alpha_r P_{t_r}$ — would close the oracle outright; it does not fit the referee’s budget today, and it is the sharpest candidate for future work.
4. **A one-round rotation edge.** A suppressor whose disclosure happens to be ordered after the honest player’s can dodge a dispute for a single round of fleeting edge-of-range contact.
5. **Privacy measured pairwise.** The deployed implementation runs the exchange at up to four participants — each round every participant publishes one response per peer, and the dispute adjudication names its pair explicitly — at per-round cost linear in the number of pairs. The anonymity-set measurements of §6, however, are two-participant; more observers can only shrink the hider’s candidate set, and we have not yet quantified by how much.
6. **Cost.** The per-round curve arithmetic is a fraction of a turn-based beat. Precomputing the position elements of (3) per map trims it by roughly a seventh at the median visible set (a quarter at its largest); the pad of (5) cannot be precomputed, by design (§4.1), which is what removed the halving an earlier design bought here. Much shorter beats would need batching or a faster field implementation.

7. **The leak is a dial, not zero.** The accumulated-negative constraint is inherent to guaranteed encounters, and we chose to measure the resulting anonymity set rather than claim more than the construction gives.

Acknowledgments

Edward Thomson’s open-source PSI implementation [13] was the practical starting point of this work: a readable, working embodiment of the classical protocol that let us prototype in days and decide what a consensus setting had to change. The protocol itself stems from four decades of prior art beginning with Meadows [1]; our contribution is the adaptation, not the trick.

References

- [1] C. Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *IEEE Symposium on Security and Privacy*, 1986.
- [2] B. A. Huberman, M. Franklin, and T. Hogg. Enhancing privacy and trust in electronic communities. In *ACM Conference on Electronic Commerce (EC)*, 1999.
- [3] R. Fagin, M. Naor, and P. Winkler. Comparing information without leaking it. *Communications of the ACM*, 39(5), 1996.
- [4] A. Shamir, R. L. Rivest, and L. M. Adleman. Mental poker. Technical Report MIT-LCS-TM-125, Massachusetts Institute of Technology, 1979.
- [5] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *EUROCRYPT*, 2004.
- [6] R. Agrawal, A. Evfimievski, and R. Srikant. Information sharing across private databases. In *ACM SIGMOD International Conference on Management of Data*, 2003.
- [7] S. Jarecki and X. Liu. Efficient oblivious pseudorandom function with applications to adaptive OT and secure computation of set intersection. In *Theory of Cryptography Conference (TCC)*, 2009.
- [8] S. Jarecki and X. Liu. Fast secure computation of set intersection. In *Security and Cryptography for Networks (SCN)*, 2010.
- [9] E. De Cristofaro and G. Tsudik. Practical private set intersection protocols with linear complexity. In *Financial Cryptography and Data Security*, 2010.
- [10] A. Narayanan, N. Thiagarajan, M. Lakhani, M. Hamburg, and D. Boneh. Location privacy via private proximity testing. In *Network and Distributed System Security Symposium (NDSS)*, 2011.
- [11] K. Thomas et al. Protecting accounts from credential stuffing with password breach alerting. In *USENIX Security Symposium*, 2019.
- [12] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe. The Apple PSI system. Apple Inc. technical report, 2021.

- [13] E. A. Thomson. Private-Set-Intersection: an ECDH-based PSI implementation. <https://github.com/EdwardAThompson/Private-Set-Intersection>, accessed July 2026.
- [14] E. Bursztein, M. Hamburg, J. Lagarenne, and D. Boneh. OpenConflict: preventing real time map hacks in online games. In *IEEE Symposium on Security and Privacy*, 2011.
- [15] E. A. Thomson. Preventing cheaters in fog of war games. <https://edward-thomson.medium.com/preventing-cheaters-in-fog-of-war-games-69f202fbe107>, June 2020.
- [16] Xaya Developers. fog-of-war: a blinded sighting test, reference implementation. <https://github.com/xaya/fog-of-war>, 2026.
- [17] D. J. Bernstein. Curve25519: new Diffie–Hellman speed records. In *Public Key Cryptography (PKC)*, 2006.
- [18] D. J. Bernstein, M. Hamburg, A. Krasnova, and T. Lange. Elligator: elliptic-curve points indistinguishable from uniform random strings. In *ACM Conference on Computer and Communications Security (CCS)*, 2013.
- [19] A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, 2016.
- [20] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood. Hashing to elliptic curves. RFC 9380, 2023.
- [21] D. Kraft. Game channels for trustless off-chain interactions in decentralized virtual worlds. *Ledger*, 1:84–98, 2016. <https://doi.org/10.5195/ledger.2016.15>.
- [22] Dark Forest team. Dark Forest: a zkSNARK-based incomplete-information game on Ethereum. <https://zkga.me>, 2020.